Anleitung

Kurzanleitung für die Arbeit mit dem Projekt.

1. Einmal einrichten

1. **VS Code öffnen**, dann *Datei*, dann *Ordner öffnen…* und den Ordner `Space` auswählen — dort, wo du das Paket entpackt hast, zum Beispiel:

`H:\Informatik\Space`

Wichtig: den **ganzen Ordner** öffnen, nicht eine einzelne Datei. Nur dann findet Python die Bausteine des Projekts.

`Space` ist dein Arbeitsordner. Alles Nötige liegt darin — auch die Bibliothek `pyfoot\`, die du nie anfassen musst.

   Wenn VS Code unten rechts nach der Python-Erweiterung fragt: installieren.

   Die richtige Python-Version auswählen. Auf vielen Rechnern sind mehrere installiert, und VS Code greift nicht immer zur richtigen. Drücke `Strg` + `Umschalt` + `P`, tippe *Python: Interpreter auswählen* und nimm die Version, die in der Prüfung aus Schritt 5 unter „verwendet wird" steht.

Unten rechts in der Statusleiste siehst du jederzeit, welche gerade ausgewählt ist.

4. Ein Terminal öffnen: *Terminal*, dann *Neues Terminal*. Prüfe die Zeile links vom Eingabecursor — dort muss `...\Space` stehen. Falls nicht, wechsle dorthin — mit deinem Speicherort statt `H:\Informatik`:

```
bash cd H:\Informatik\Space
```

5. Prüfen, ob alles Nötige vorhanden ist:

```
bash python tools\check_environment.py
```

Meldet die Prüfung etwas als fehlend, wende dich an den IT-Support — die Installation braucht Rechte, die ein normales Konto nicht hat. Der Support findet die Anleitung dazu in `SUPPORT_INSTALLATION.md`.

Fehlen Pakete, **fragt** die Prüfung, ob sie sie gleich installieren soll. An der Schule mit `n` antworten; auf dem eigenen Rechner darfst du mit `j` bestätigen, danach prüft sie von selbst noch einmal (siehe die erste Hausaufgabe, `AB01d`).

Auf dem eigenen Rechner hilft bei Problemen mit Python ein Doppelklick auf `tools\setup_python.bat`. Das Skript schaut, ob `python` den Microsoft Store öffnet, statt Python zu starten. Es trägt Python in den PATH deines Benutzerkontos ein und installiert die fehlenden Pakete. Danach VS Code schließen und neu öffnen. Nur prüfen, ohne etwas zu ändern:

```
bash tools\setup_python.bat /check
```

6. Testen, ob alles läuft:

```
bash python main_space.py
```

Es öffnet sich ein Fenster, in dem ein Raumschiff losfliegt. Schließen mit `Esc` oder über das Fensterkreuz.

> Deine Lösungen musst du nicht installieren — du schreibst sie einfach in den Ordner `Space` und startest sie dort.

> Die Befehle in dieser Anleitung setzen voraus, dass das Terminal im Ordner `Space` steht. Ausnahme: Die Skripte im Ordner `tools\` finden das Projekt selbst und lassen sich von überall aufrufen.

2. Wo liegt was

Alle Angaben ab hier beziehen sich auf den Projektordner `Space`: `ships\normal_spaceship.py` meint die Datei `normal_spaceship.py` im Ordner `ships` darin — egal, wo `Space` bei dir liegt.

Ordner	Inhalt
`ships\`	**deine** Raumschiffe — hier fängst du an
`levels\`	**deine** Welten — je eine Datei, `Level0` bis `Level5…`
`space\`	der Kursinhalt: PowerUps, Asteroiden, die vorgegebenen Welten
`pyfoot\`	die Bibliothek dahinter — hier musst du nichts ändern
`tools\`	Prüfwerkzeuge
`main_space.py`	ein fertiges Beispiel zum Abgucken
`main_editor.py`	dasselbe, aber mit Bedienleiste und Klassenanzeige

Die ersten beiden Ordner gehören dir. Alles andere ist vorgegeben — dort musst du nichts ändern, und die Oberfläche lässt dich dort auch nichts löschen.

In `ships\normal_spaceship.py` liegt dein erstes Raumschiff. Es ist mitgeliefert, aber ab sofort **deine** Datei: Schreib deine ersten Anweisungen einfach hinein.

Die Arbeitsblätter liegen nicht hier — du bekommst sie von deiner Lehrkraft, meist als PDF.

3. Programme starten

Zum Starten gibt es zwei Wege:

Der Play-Knopf oben rechts im Editor (das Dreieck). Er führt die Datei aus, die du gerade offen hast. Bequem, aber er benutzt die Python-Version, die VS Code ausgewählt hat — siehe Schritt 3 der Einrichtung.

Das Terminal:

```
python main_space.py
```

> **Läuft es im Terminal, aber der Play-Knopf meldet `ModuleNotFoundError: No module named 'pygame'`?** Dann hat VS Code eine andere Python-Version ausgewählt als dein Terminal benutzt. Auswahl umstellen wie in Schritt 3 der Einrichtung beschrieben.

Mit `F5` startest du dieselbe Datei im Debugger — dort kannst du Haltepunkte setzen und Schritt für Schritt zusehen.

Mit Bedienleiste starten

Unter dem Fenster lässt sich eine Leiste einblenden, mit der du das Programm anhalten und Schritt für Schritt weiterlaufen lassen kannst.

Am einfachsten: ein fertiges Beispiel mit der Startwelt der ersten Aufgaben. Es schaltet die Leiste selbst ein — auch über den Play-Knopf:

```
python main_editor.py
```

Für dein eigenes Programm genügt eine Zeile im Terminal:

```
$env:PYFOOT_UI = "1"
```

Danach zeigt jedes Programm die Leiste, solange **dieses** Terminal offen ist. Ein neues Fenster und der Play-Knopf kennen sie nicht wieder — das ist die häufigste Stolperfalle. Wieder abschalten:

```
$env:PYFOOT_UI = ""
```

Prüfen, ob sie gesetzt ist:

```
echo $env:PYFOOT_UI
```

Schaltfläche	Taste	Wirkung
Start / Pause	`Leertaste`	anhalten und fortsetzen — auch mittendrin
Durchlauf	`D`	das Programm starten bzw. einen ganzen Durchlauf ausführen
Schritt	`S`	genau **eine** Anweisung ausführen
Zurücksetzen	`R`	zurück auf Anfang
Tempo	`+` / `-`	schneller oder langsamer

Ist eine Welt größer als das Fenster — die großen PowerUp-Felder sind es —, erscheinen am Rand **Bildlaufleisten**. Rollen kannst du mit dem Mausrad (mit `Umschalt` seitwärts), durch Ziehen des Schiebers oder mit einem Klick auf die Leiste.

Was du auslöst, wird unten in der Leiste gemeldet: **grün mit Haken**, wenn es geklappt hat — etwa nach *Welt sichern* —, orange bei einer Warnung und rot bei einem Fehler.

Der Knopf **Schritt** ist der nützlichste: Damit siehst du deinem Programm Anweisung für Anweisung zu und erkennst genau, wo es aus dem Tritt gerät. Ausgegraute Knöpfe bewirken gerade nichts — die Leiste zeigt dir also selbst, was möglich ist. Rechts neben den Klassennamen steht unter **Taste**, mit welcher Ziffer du die jeweilige Klasse auswählst.

Rechts steht der **Klassenbaum**. Darüber wechselst du die Welt, legst neue Klassen an und setzt Objekte — dazu gleich mehr.

4. Eine Aufgabe lösen

Deine Anweisungen schreibst du in die Datei `ships\normal_spaceship.py`. **Diese Datei gehört dir.** Sie sieht am Anfang so aus:

```
class NormalSpaceship(Spaceship):
   __slots__ = ()

   def init(self) -> None:
       # Hier stehen deine Anweisungen, eine pro Zeile.
```

```
        self.move()
        self.drop_power_up()
        self.move()
        self.turn_left()
```

Die Oberfläche musst du nach einer Änderung **nicht** neu starten:

1. Datei speichern — VS Code tut das von selbst, sobald du ins Fenster wechselst.
2. Im Fenster **Zurücksetzen** drücken (`R`).
3. **Start**.

Beim Zurücksetzen liest der Editor deine geänderten Dateien neu ein. Unten in der Bedienleiste steht dann zum Beispiel `Neu eingebunden: normal_spaceship.py` — daran siehst du, dass deine Änderung angekommen ist.

Drei Dinge, die immer gleich sind:

- Deine Klasse erbt von `Spaceship` (oder von `SensorSpaceship`, wenn dein Schiff seine Umgebung wahrnehmen soll).
- Deine Anweisungen stehen in der Methode `init`. Sie wird beim Start **einmal** ausgeführt, von oben nach unten.
- Welche Welt du brauchst, steht in der Aufgabe. **Rechtsklick** auf ihren Namen im Klassenbaum, dann `Level1PowerUpRow() anzeigen`. Danach setzt du dein Raumschiff hinein (siehe unten) — nur `Level0` bringt eines mit.
- Wohin, zeigt die Welt selbst: Das **Startfeld** hat einen blauen Rahmen mit einem Pfeil nach rechts. Dorthin kommt dein Raumschiff, und in diese Richtung schaut es am Anfang.

Eigene Methoden darfst du jederzeit ergänzen. Das ist sogar erwünscht: Sobald du dieselben Zeilen mehrfach schreibst, lohnt sich eine eigene Methode.

```
class NormalSpaceship(Spaceship):
   __slots__ = ()

   def init(self) -> None:
       self.fly_and_drop()
       self.fly_and_drop()

   def fly_and_drop(self) -> None:
       self.move()
       self.drop_power_up()
```

Ein zweites Raumschiff

Für ein Raumschiff mit anderem Verhalten brauchst du keine neue Datei von Hand: **Rechtsklick** auf `Spaceship` im Klassenbaum, dann *Unterklasse anlegen…*. Die Oberfläche legt die Datei in `ships\` an und trägt sie in `ships\__init__.py` ein — so ist sie auch nach einem Neustart wieder da. Für eine neue Welt geht es genauso mit `StartWorld`; sie landet in `levels\`.

Lieber von Hand? Eine Datei, die du selbst in VS Code anlegst, liest das Programm erst ein, wenn sie in der `__init__.py` ihres Ordners steht. Für eine Welt `levels\meine_welt.py` mit der Klasse `MeineWelt` sind das zwei Zeilen in `levels\__init__.py`:

```
from .meine_welt import MeineWelt      # bei den anderen Imports

__all__ = [
   ...
   "MeineWelt",                        # in die Liste
]
```

Speichern, im Fenster `R` — dann steht die Welt im Klassenbaum. Fehlt der Eintrag, sagt dir die Bedienleiste nach `R`:

```
Nicht eingebunden: levels/meine_welt.py -- Eintrag in __init__.py fehlt.
```

Damit dein neues Raumschiff auch fliegt, musst du eines davon in die Welt setzen: **Rechtsklick** auf seinen Namen, dann `MeinSchiff() erzeugen` und mit der Maus auf das Startfeld ziehen. In `Level0` steht bereits ein Raumschiff — das nimmst du per **Rechtsklick** darauf und *Entfernen* heraus; das Tastenkürzel dafür ist `Entf`.

5. Die wichtigsten Befehle

Für jedes Raumschiff:

Befehl	Wirkung
`self.move()`	ein Feld vorwärts fliegen
`self.turn_left()`	eine Vierteldrehung nach links
`self.collect_power_up()`	PowerUp vom eigenen Feld aufsammeln
`self.drop_power_up()`	PowerUp auf dem eigenen Feld ablegen
`self.say("Hallo")`	Text anzeigen und ausgeben
`self.power_up_count`	wie viele PowerUps an Bord sind

Zusätzlich nur für `SensorSpaceship`:

Befehl	Antwort
`self.can_move()`	Ist das Feld vor mir frei?
`self.is_power_up_here()`	Liegt hier ein PowerUp?
`self.is_facing_north()`	Schaue ich nach Norden? (auch `south`, `east`, `west`)

6. Der Editor hilft mit

VS Code prüft deinen Code schon beim Schreiben und unterkringelt Stellen, die nicht zusammenpassen — etwa wenn du einer Zahl einen Text zuweist oder eine Methode mit der falschen Anzahl an Werten aufrufst. Fahre mit der Maus über die Kringel, dann erscheint die Erklärung.

Diese Hinweise **verhindern das Starten nicht**. Dein Programm läuft trotzdem. Sie sind ein Angebot: Meistens steckt dort schon der Fehler, den du sonst erst beim Ausführen findest.

Deshalb lohnt es sich, die Typen mitzuschreiben — das `-> None` hinter `init` ist genau so eine Angabe:

```
def fly_and_drop(self) -> None:
    ...

def count_steps(self) -> int:
    ...
```

Dieselbe Prüfung gibt es auch auf der Konsole, zum Beispiel wenn du vor der Abgabe alles auf einmal durchsehen willst:

```
python tools\check_code.py
```

Das prüft alle deine Lösungsdateien. Für eine einzelne Datei:

```
python tools\check_code.py ships\normal_spaceship.py
```

Die Ausgabe zeigt Datei, Zeile und die Stelle im Code. Auch hier gilt: Die Meldungen verhindern nichts, sie sind ein Hinweis.

7. Wenn etwas schiefgeht

- **Das Fehlerfenster erscheint** — dein Programm hat etwas versucht, was nicht geht: aus der Welt hinausfliegen, gegen einen Asteroiden fliegen, ein PowerUp aufsammeln, wo keines liegt — oder im Code steckt ein Fehler, etwa ein Tippfehler im Namen. Das Programm hält an, das

Fenster bleibt offen.

Taste	Wirkung
`E`	**Zur Zeile** — öffnet VS Code an der Stelle in deiner Datei
`R`	**Zurücksetzen** — liest geänderte Dateien neu ein und beginnt von vorn
`Esc`	schließt nur das Fehlerfenster

Also: Fehler lesen, `E`, Zeile korrigieren, zurück ins Fenster, `R`, Start. Dieselbe Meldung steht auch im Terminal; dort führt ein Strg+Klick auf `ships/normal_spaceship.py:12` direkt zur Zeile.

Bleibt das Fehlerfenster nach `R` stehen, ist die Datei noch nicht in Ordnung — dann läuft der alte Stand weiter, bis der Fehler behoben ist. - **Der Editor startet gar nicht, im Terminal steht „Fehler vor dem Start“** — eine deiner Dateien lässt sich nicht lesen, meist fehlt eine Klammer oder ein Doppelpunkt. Unter `Stelle:` stehen Datei und Zeile. - **`ModuleNotFoundError: No module named 'pyfoot'`** — das Terminal steht im falschen Ordner. Siehe Schritt 4 der Einrichtung. - **Der Play-Knopf meldet ein fehlendes Paket, das Terminal aber nicht** — VS Code hat eine andere Python-Version ausgewählt. Siehe Schritt 3 der Einrichtung.

Ein häufiger Sonderfall: Im Projektordner liegt ein Ordner `.venv`. Das ist eine eigene, abgeschottete Python-Umgebung. Hat VS Code diese ausgewählt, aber es wurde nie etwas hineininstalliert, fehlt dort alles. Entweder eine andere Version wählen oder die Pakete dort nachinstallieren:

```
bash .venv\Scripts\python.exe -m pip install pygame mypy pytest
```

- **PowerShell fragt: „Führen Sie ausschließlich vertrauenswürdige Skripts aus"** — Windows markiert alles, was aus dem Internet kommt. Betrifft meist VS Code selbst, wenn es als ZIP heruntergeladen wurde. Die Markierung lässt sich entfernen (der Pfad steht in der Warnung):

```
bash Unblock-File -Path "PFAD\AUS\DER\WARNUNG.ps1"
```

Das braucht keine Administratorrechte. - **`Duplicate module named "__main__"`** bei einer Typprüfung — ebenfalls der falsche Ordner. Diese Meldung von mypy ist irreführend, gemeint ist „Pfad nicht gefunden". Die Skripte unter `tools\` funktionieren dagegen von überall. - **Das Fenster reagiert nicht** — vermutlich eine Endlosschleife. Fenster schließen, Schleifenbedingung prüfen. - **Zu schnell oder zu langsam?** Vor `run()` einfügen:

`python from pyfoot import get_engine get_engine().step_duration = 0.4 # Sekunden pro Schritt`

8. Eine neue Version holen

Im Laufe des Kurses kommen neue Welten und neue Raumschiffe dazu. Du musst dafür **nicht** neu anfangen — und deine Arbeit geht dabei nicht verloren.

1. Sieh im Terminal erst nach, was passieren würde:

`bash python tools\update_space.py --dry-run`

Das Werkzeug holt die neueste Version selbst von GitHub. Dazu braucht es das Programm `git`, eine Anmeldung ist nicht nötig. Hast du schon die neueste Version, sagt es das und tut nichts.

2. Wenn das passt, ohne `--dry-run` noch einmal:

`bash python tools\update_space.py`

Was dabei mit deinen Dateien geschieht:

Deine Datei	Was passiert
Du hast sie geändert	Sie **bleibt so, wie sie ist.** Die neue Version liegt danach als `name.py.neu` daneben — zum Vergleichen.
Du hast sie nie angefasst	Sie wird durch die neue ersetzt.
Sie gibt es nur bei dir	Sie bleibt unberührt. Deine eigenen Raumschiffe und Welten gehen nie verloren.
Sie ist neu im Kurs	Sie kommt dazu.

Die beiden Dateien `ships\__init__.py` und `levels\__init__.py` sind ein Sonderfall: Dort steht Mitgeliefertes **und** Eigenes. Sie werden zusammengeführt, damit neue Raumschiffe im Klassenbaum auftauchen und deine eigenen dort bleiben.

Bevor irgendetwas geändert wird, landet ein **Backup** deines ganzen Ordners als ZIP-Datei unter `backup\`. Falls doch etwas schiefgeht, packst du sie einfach wieder aus.

Kommt das Werkzeug nicht an GitHub heran — kein `git`, kein Netz —, nennt es den Grund. Dann geht es auch mit `Space.zip` aus Moodle: Datei herunterladen, **nicht** auspacken, und den Pfad dazu angeben:

```
python tools\update_space.py C:\Users\DEINNAME\Downloads\Space.zip
```

> **Sucht das Werkzeug nur nach `Space.zip` und fragt nicht bei GitHub?** Dann hat dein Projekt noch eine ältere Fassung. Frische einmal mit `Space.zip` auf wie oben — dabei kommt auch die neue Fassung des Werkzeugs. Danach geht es über GitHub.

> **Fehlt `tools\update_space.py` ganz**, packe `Space.zip` einmal irgendwo aus und kopiere daraus die Datei `tools\update_space.py` in deinen Projektordner nach `tools\`.

Für die Lehrkraft

Pfadangaben: Die Projekte liegen nebeneinander in einem gemeinsamen Überordner, etwa `SAE-GMO\pyfoot-space` und `SAE-GMO\pyfoot-course`. Befehle gelten ab dem jeweils genannten Projektordner. Wo die Nachbarn liegen, steht je Projekt an genau einer Stelle: `pyproject.toml`, Abschnitt `[tool.sae-gmo.neighbours]`.

Nach dem Klonen von pyfoot-space fehlt die Bibliothek; sie liegt nicht im Repository, sondern wird in der festgelegten Version geholt:

```
python tools\get_pyfoot.py
```

Im Ordner `pyfoot-space` — Tests und Typprüfung des Projekts:

```
python -m pytest
```

```
python tools\check_project.py
```

Im Ordner `pyfoot-course` (privat, auf Anfrage) — Arbeitsblätter, Musterlösungen und Zielbilder. `python -m pytest` prüft dort auch, ob Arbeitsblätter und Code zusammenpassen:

```
python tools\build_docs.py
```

Das Paket für die Schüler:innen bauen — vor jedem Hochladen in Moodle:

```
python tools\build_student_package.py
```

Es legt `dist\Space.zip` an, mit PyFoot in der festgelegten Version, aber **ohne `tests\`** und ohne die Werkzeuge, die nur du brauchst. Die Musterlösungen liegen ohnehin nicht in Space, sondern in `pyfoot-course`; ein Test prüft trotzdem, dass keine mitgeht.

Der Text für die Moodle-Ankündigung steht fertig in `MOODLE_UPDATE.md` — in zwei Varianten, für das erste Mal und für jedes weitere.

Jedes Paket bekommt dabei ein Manifest (`tools\paket.json`) mit Datum und einer Prüfsumme je Datei. Davon lebt Abschnitt 8: Nur Dateien, deren Prüfsumme noch zu der zuletzt ausgelieferten Version passt, dürfen überschrieben werden. Ein Paket, das ohne dieses Werkzeug entstanden

ist, hat kein Manifest — dann bleibt beim Auffrischen vorsichtshalber **jede** abweichende Datei stehen.

> Lade **nie** den Projektordner selbst hoch, sondern immer das Paket. Nur das Paket hat das Manifest, und nur darin liegt PyFoot sicher in der richtigen Version.

Zielbild eines Arbeitsblatts aus einer Musterlösung erzeugen (im Ordner `pyfoot-course`):

```
python tools\render_figure.py test_block01:WaveLineFigure bild.png
```

Klassendiagramm des Projekts erzeugen (rein lokal, kein Datenversand):

```
python -c "from pyfoot import write_diagram as w; print(w(root='.', ignore={'tests'}))"
```

Bewusst nicht vorhanden: Einige Methoden fehlen absichtlich, weil ihre Implementierung Gegenstand einer Aufgabe ist — etwa `turn_right()` (Einstieg Methoden) sowie `is_asteroid_left()` und `is_asteroid_right()` (logische Verknüpfungen). Tests sichern ab, dass sie nicht versehentlich ergänzt werden.

Welten mit der Maus bauen

Mit eingeschalteter Oberfläche (`$env:PYFOOT_UI = "1"`) lässt sich eine Welt umbauen, statt jedes Feld im Quelltext einzutragen.

Bedienung	Taste	Wirkung
Bearbeiten	`B`	Bearbeitungsmodus ein und aus
Klick auf eine Klasse	`1` bis `9`	auswählen, was gesetzt wird
Klick auf ein leeres Feld	—	Objekt setzen; die Auswahl bleibt
`Umschalt` + Klick	—	auch auf ein belegtes Feld setzen
Objekt anklicken und ziehen	—	verschieben
Doppelklick auf eine Klasse	—	ihre Datei im Editor öffnen

Rechtsklick	—	Kontextmenü öffnen
—	`Entf`	ausgewähltes Objekt entfernen
Welt sichern	`W`	Aufbau in den Quelltext schreiben

Welt sichern schreibt den Aufbau als Methode `prepare()` in die Datei der Welt — etwa `levels\level1_power_up_row.py`. Alles außerhalb dieser Methode bleibt stehen, fehlende `import`-Zeilen kommen dazu, und vor jedem Schreibvorgang entsteht eine Sicherungskopie `level1_power_up_row.py.bak`. Gelöschte Objekte fehlen danach ebenso in der Datei wie neue darin stehen. Mit **Zurücksetzen** (`R`) baut sich die Welt aus der gesicherten Datei neu auf — ohne Neustart.

Vier Dinge, die man wissen sollte:

- **Aus Schleifen werden Listen.** Die Welten des Kurses bauen sich mit Schleifen auf. Beim Sichern entsteht daraus eine flache Liste von `add_object`-Zeilen — bei einer 25 × 25-Welt sind das 625 Zeilen. Deshalb warnt der erste Druck auf **Welt sichern** und nennt die Zeilenzahl; erst der zweite schreibt wirklich.
- **Das Raumschiff von `Level0` wird nicht mitgeschrieben**, sondern als `START` vermerkt — es steht im Konstruktor, nicht in `prepare()`. Verschiebst du es, ändert sich die `START`-Zeile. Genau dafür ist die Funktion gedacht: Ein falsch gesetztes Startfeld hat Aufgabe 5b schon einmal unlösbar gemacht. **Löschst** du es, verschwindet es beim Sichern auch aus dem Konstruktor.
- **Von Hand geändert?** Hast du die Datei der Welt in VS Code bearbeitet, warnt der erste Druck auf **Welt sichern**: Der Aufbau im Fenster kennt deine Änderung nicht und würde sie überschreiben. `R` übernimmt sie stattdessen.
- **Der Zufall bleibt Zufall.** Aus `RandomAsteroid(0.5)` wird wieder `RandomAsteroid(0.5)`, nicht das gerade gewürfelte Ergebnis. Sonst läge die Lücke in der Asteroidenwand künftig immer an derselben Stelle.

Die Meldung nach dem Sichern steht unten in der Bedienleiste.

Die Kontextmenüs

Ein **Rechtsklick** öffnet ein Menü. Jeder Eintrag hat eine Ziffer, `Esc` schließt es wieder — das Fenster bleibt dabei offen.

Auf einer Klasse in der rechten Liste:

Eintrag	Wirkung

`PowerUp()` erzeugen	setzt ein einzelnes Objekt in die Welt
`Level0()` anzeigen	**bei einer Welt:** wechselt zu dieser Welt
Quelltext öffnen	öffnet die Datei in VS Code, an der richtigen Zeile
Unterklasse anlegen…	fragt nach dem Namen und legt eine neue Datei an
Bild zuweisen…	zeigt die Bilder aus `space\assets\images\`
Löschen	entfernt eine Klasse, die du selbst angelegt hast

Der erste Eintrag zeigt genau den Ausdruck, den man auch selbst schreibt: `PowerUp` ist der Typ, `PowerUp()` ein einzelnes Objekt — die Unterscheidung aus `AB02b_Infoblatt_Klassen`.

Welt wechseln: Ein Rechtsklick auf eine Welt in der Liste zeigt diese Welt an — praktisch, um im Unterricht schnell zwischen den Aufgabenwelten zu springen. Der Name steht danach in der Titelzeile.

> Das Fenster lässt sich ziehen und maximieren. Eine selbst gewählte Größe bleibt beim Wechseln erhalten — für den Beamer also einmal groß ziehen und dann durch die Welten springen.

> `Level0` ist die einzige Welt, die ein Raumschiff mitbringt. Sie bindet `ships\normal_spaceship.py` oben bei den anderen Imports ein — so, wie es die Schüler:innen lernen sollen. Steckt in einer Datei unter `ships\` ein Fehler, meldet sich das beim Start im Terminal und im laufenden Editor im Fehlerfenster.

> Das Raumschiff kommt dabei **nicht** mit. Jede Welt bringt ihr eigenes mit, so wie es in ihrem Quelltext steht. Genau darum geht es später: das richtige Schiff in die richtige Welt einzusetzen.

Löschen geht nur bei Klassen, die in `ships\` oder `levels\` liegen — also bei deinen eigenen. Der Kursinhalt lässt sich nicht löschen; der Eintrag ist dort ausgegraut. Der erste Klick fragt nach, der zweite führt aus. Die Datei wird dabei nicht wirklich gelöscht, sondern in `<name>.py.bak` umbenannt, und ihr Eintrag in der `__init__.py` des Ordners verschwindet.

Auf einem Objekt in der Welt: seine Befehle, dazu **Inspizieren**. Ein Klick auf einen Befehl führt ihn aus und zeigt das Ergebnis in der Bedienleiste — praktisch zum Vorführen im Unterricht. Schlägt der Befehl fehl, erscheint das Fehlerfenster; das Programm läuft weiter.

> Angezeigt wird nur, was die Klasse wirklich hat. `turn_right()` taucht dort also **nicht** auf — es ist Gegenstand einer Aufgabe.

Inspizieren zeigt den Zustand eines Objekts — alle seine Werte auf einmal:

```
NormalSpaceship — Zustand
 _power_ups                5000
 _rotation                    0
 _x                           0
 _y                           5
```

Die Anzeige aktualisiert sich laufend, und die Bedienleiste bleibt bedienbar. **Damit lässt sich der beste Trick vorführen:** Anzeige offen lassen und mit `S` Schritt für Schritt weitergehen — dann sieht man, wie sich `_x`, `_y` und `_power_ups` bei jeder Anweisung ändern. `Esc` schließt sie wieder.

Farbige Werte lassen sich anklicken und ändern — zum Beispiel die Wahrscheinlichkeit eines `RandomPowerUp`. Änderbar ist nur, was beim **Welt sichern** auch im Quelltext landet; alles andere wäre beim nächsten Zurücksetzen ohnehin wieder weg. Die Lage eines Objekts ändert man durch Ziehen, nicht durch Eintippen.

Beim **Erzeugen** eines Objekts fragt die Oberfläche nach den Werten, wenn der Konstruktor welche kennt — vorbelegt mit den Vorgaben:

```
RandomPowerUp(probability) — Werte:  [0.5]
```

Eingabetaste übernimmt die Vorgabe, `0.9` erzeugt stattdessen `RandomPowerUp(0.9)`.

Unterklasse anlegen erzeugt eine Datei, die sofort läuft:

```
class MeinSchiff(Spaceship):
   """Beschreibung."""

   def init(self) -> None:
       """Hier stehen die Anweisungen."""
```

Sie liegt in `ships\` (Akteure) oder `levels\` (Welten), steht **sofort** in der Klassenliste und ist in der `__init__.py` des Ordners eingetragen — also auch nach einem Neustart wieder da.